거래 수정 요청은 둘 다 성공했는데, 왜 보유 수량은 틀렸을까?

DAYNOMY에서 원장 조회 전 포트폴리오 잠금으로 정합성을 지킨 과정

카야(choiyoung69) · 2026년 11월 10일 최종 수정

1. 성공 응답 뒤에 남은 2주의 차이

10주 매수 기록 두 건이 있는 포트폴리오를 생각해 보자. 현재 보유 수량은 20주다. 첫 번째 거래를 12주로, 두 번째 거래를 20주로 수정했다면 최종 보유 수량은 32주여야 한다. DAYNOMY의 동시성 재현 테스트에서는 두 요청이 모두 HTTP 200을 반환했지만, 저장된 현재 보유는 30주였다. 거래 이력을 다시 읽어 계산하면 32주가 나왔다. 성공 응답과 데이터의 정합성이 서로 다른 결과를 가리킨 것이다.[1]

DAYNOMY는 사용자가 보유 자산과 거래를 기록하고 자산 변화를 확인하는 서비스다. 거래 기록은 현재 보유 수량과 과거 평가금액을 계산하는 입력이 된다. 따라서 거래 수정 한 건을 저장하는 일에는 여러 데이터의 관계를 유지하는 책임도 따른다. 이력을 정상적으로 수정했더라도 그 결과를 보여 주는 보유 수량이 다르면 사용자는 어느 값을 믿어야 할지 알 수 없다.

기존에는 스냅샷을 갱신할 때 포트폴리오 행을 잠갔다. 그때는 두 번째 요청이 이미 첫 번째 거래의 예전 수량인 10주를 읽은 뒤였다. 잠금을 원장 조회 전으로 옮기자 두 번째 요청은 첫 번째 요청이 커밋한 12주를 읽고 자신의 변경분과 합산해 32주를 저장했다.[1][3]

이 글에서는 이 문제를 실제 요청과 PostgreSQL로 재현한 방법, 비관적 잠금을 원장 조회 앞으로 옮긴 이유, 여러 변경 경로를 같은 규약으로 묶은 과정을 설명한다. Java·Spring Boot·JPA로 CRUD를 구현해 보았고 트랜잭션의 기본 역할을 아는 독자를 대상으로 한다. 잠금의 문법보다, 서비스에서 지켜야 할 데이터 관계를 잠금의 대상과 시점으로 연결하는 과정에 초점을 둔다.

본문의 변경 전후 수치와 테스트 개수는 2026년 10월 19~20일 ADR에 남긴 실행 기록이다. 최종본은 ADR·수정 커밋·관련 소스를 대조해 정리했다. 테스트 수치는 당시 실행 기록을 인용했다.

2. 함께 맞아야 하는 세 가지 데이터

문제를 이해하려면 원장, 현재 보유, 과거 스냅샷의 역할을 구분해야 한다. DAYNOMY의 포트폴리오에는 다음 데이터가 연결되어 있다.[1][3]

데이터	역할	이 사례의 정상 결과
거래·보유 변경 이력	매수·매도·정정 등의 입력 기록	12주 매수, 20주 매수
현재 보유	이력을 재생한 현재 수량과 평균 매입가	32주
일일 스냅샷	특정 날짜의 평가금액과 손익	종가 100원이면 3,200원

거래 이력은 PortfolioHoldingHistory, 현재 보유는 PortfolioHolding, 날짜별 평가는 PortfolioDailySnapshot으로 저장한다. PortfolioLedgerCalculator는 정렬된 이력을 재생해 수량과 원가·실현손익을 계산한다. 서비스는 그 계산 결과를 현재 보유에 반영하고, 과거 거래 수정의 영향을 받는 날짜부터 스냅샷을 다시 계산한다.

현재 보유를 별도로 저장하면 매번 모든 이력을 재생하지 않고 조회할 수 있다. 대신 이력과 현재 보유를 함께 맞춰야 한다. 이 사례에서 지킬 조건은 “같은 포트폴리오에 대해 커밋된 이력을 재생한 수량과 저장된 현재 수량이 같아야 한다”이다. 가격이 준비된 재현 날짜에서는 스냅샷의 평가금액도 그 수량과 맞아야 한다. 가격·환율이 누락된 경우의 보존 정책은 뒤에서 별도로 다룬다.

한 요청 안에서 이력 변경과 보유·스냅샷 갱신을 트랜잭션으로 묶는 일은 필요하다. 중간에 실패하면 일부 변경만 남지 않게 해야 하기 때문이다. 하지만 여러 요청이 같은 포트폴리오를 읽는 순서까지 그것만으로 정해지지는 않는다. 두 요청이 각각 이전 상태를 읽고 서로 다른 계산 결과를 저장하면, 각 트랜잭션이 성공적으로 끝나도 데이터 사이의 관계가 어긋날 수 있다.

보호할 대상을 거래 한 행으로만 잡기도 어렵다. 두 요청은 서로 다른 거래를 수정하지만, 두 거래의 합으로 같은 현재 보유와 스냅샷을 만든다. 충돌의 기준은 수정 대상 거래의 ID가 같은지가 아니라 계산에 사용하는 원장과 결과를 공유하는지였다. 이 때문에 이번 문제의 공통 보호 단위를 포트폴리오로 잡았다.

3. 스냅샷 저장 시점의 포트폴리오 잠금이 부족했던 이유

변경 전 거래 수정·삭제는 이력 조회 → 이력 변경 → 원장 재생과 현재 보유 반영 → 포트폴리오 행 잠금 → 스냅샷 갱신 순서였다. 아래 표는 두 요청이 어떤 이력을 읽어 30주를 저장했는지 보여 준다.[1][2][3]

흐름	첫 거래 수정 요청 A	둘째 거래 수정 요청 B
기존 상태	첫 거래 10주, 둘째 거래 10주	첫 거래 10주, 둘째 거래 10주
겹친 요청	첫 거래를 12주로 변경	A의 커밋 전 첫 거래 10주를 읽음
A 완료	자신의 변경을 반영해 커밋	읽은 이력을 가진 채 대기
B 계산	완료	예전 첫 거래 10주 + 자신의 둘째 거래 20주
최종 저장	원장의 첫 거래는 12주	현재 보유를 30주로 저장

최종 DB의 거래 이력은 첫 거래 12주, 둘째 거래 20주다. B가 저장한 보유 수량만 예전 첫 거래를 포함한다. 이력 두 건을 새로 읽어 재생하면 32주가 나오지만 현재 보유는 30주가 되는 이유다.

여기에는 JPA의 영속성 컨텍스트도 관여한다. 한 영속성 컨텍스트는 같은 엔티티 식별자에 대해 하나의 관리 객체를 유지한다.[6] B가 A의 커밋 전에 읽은 이력 엔티티를 이미 관리하고 있다면, 나중에 조회 쿼리가 실행됐다는 사실만으로 그 객체의 상태까지 최신 값으로 교체됐다고 볼 수 없다. ADR의 재현에서는 예전 10주가 계산에 남았다. “쿼리를 다시 했으니 최신 값일 것”이라는 가정은 이 흐름의 정합성 근거가 되지 못했다.

스냅샷 저장에서 얻는 잠금은 이미 읽은 이력이나 이미 수행한 계산을 되돌리지 않는다. 마지막 저장을 순서대로 처리하더라도, 각 요청이 어떤 입력으로 계산했는지까지 보호하지 못한다. 문제를 해결하려면 이력·보유를 읽고 계산하는 구간도 공통 잠금 안에 들어와야 했다.

보유 정정·삭제에는 같은 포트폴리오 잠금이 없었고, V1 등록은 회원 잠금만 먼저 잡았다. 따라서 수정 API 하나의 잠금 위치만 확인해서는 충분하지 않았다. 같은 데이터를 바꾸는 다른 진입점까지 함께 살펴봐야 했다.

4. 어떤 요청 중첩에서 실패하는지 고정하기

동시성 문제를 고치려면 변경 전 실패를 다시 만들 수 있어야 한다. 스레드 두 개를 동시에 시작하는 것만으로는 필요한 순서가 만들어졌는지 알기 어렵다. 실행 환경에 따라 한 요청이 먼저 끝나면 잘못된 구현도 통과할 수 있다. 그래서 PortfolioConcurrencyAcceptanceTest는 실제 HTTP 요청과 PostgreSQL 16을 사용하고, 별도 DB 연결로 대기 지점을 만들었다.[2]

재현 데이터는 단순하다. 같은 종목에 10주 매수 이력 두 건, 현재 보유 20주, 종가 100원과 초기 스냅샷을 준비한다. 다음 순서로 두 요청을 겹친다.

1. 별도 연결에서 포트폴리오 행을 SELECT ... FOR UPDATE로 잠근다.
2. 첫 거래를 12주로 수정하는 요청을 보낸다.
3. DB에 잠금을 기다리는 요청이 나타났는지 확인한다.
4. 둘째 거래를 20주로 수정하는 요청을 보내고 대기 요청 증가를 확인한다.
5. 별도 연결의 잠금을 해제하고 두 응답을 받는다.
6. 저장된 이력·현재 보유·스냅샷을 새로 조회해 비교한다.

별도 연결이 잠금을 쥐고 있으면 첫 요청이 커밋하기 전에 둘째 요청을 끼워 넣을 수 있다. 같은 잠금을 미리 잡아도 구현에 따라 두 요청이 멈추는 위치가 달랐다.[1][2][3]

구현	첫 요청이 기다리는 곳	둘째 요청이 기다리는 곳
변경 전	보유 반영 뒤, 별도 연결이 잠근 포트폴리오 행	예전 이력을 읽은 뒤, 첫 요청이 갱신 중인 현재 보유 행
변경 후	이력 조회 전, 별도 연결이 잠근 포트폴리오 행	이력 조회 전, 같은 포트폴리오 행

테스트는 pg_stat_activity에서 현재 DB의 활성 요청 중 wait_event_type = 'Lock'인 개수를 20ms 간격으로 최대 10초간 확인한다. 대기 행은 변경 전후 서비스 흐름을 바탕으로 설명했다. 관측 쿼리는 테스트 DB의 대기 수를 세므로 다른 요청이 섞이지 않는 환경을 전제로 한다.

마지막 검증은 응답 코드에서 끝나지 않는다. 두 수정 요청이 200을 반환하는지 확인한 다음, 원장 재생 수량과 저장된 보유가 모두 32주인지, 스냅샷은 하나이고 평가금액은 3,200원인지 검사한다. 핵심 동시 정정 사례에는 @RepeatedTest(10)을 적용했다. 반복 횟수 자체보다 의미 있는 것은 문제가 발생하는 중첩을 먼저 만들고 결과 데이터의 관계를 검증한다는 점이다.

5. 세 가지 대안과 포트폴리오 잠금의 선택

ADR에서는 원장 조회 전 비관적 잠금, 버전 기반 충돌 감지, Serializable과 재시도를 비교했다. 기준은 일반적인 기술의 장단점보다 기존 DAYNOMY의 변경 경로를 어떻게 함께 보호할 수 있는지였다.[1]

대안	이번 문제에서 확인할 것	선택에 영향을 준 비용
포트폴리오 비관적 잠금	이력 조회 전 같은 부모 행을 잠그는가	같은 포트폴리오 요청의 대기
버전 기반 충돌 감지	자식 이력 변경도 공통 버전 충돌로 잡는가	버전 갱신·충돌 응답·재시도 설계
Serializable과 재시도	읽기·쓰기 충돌을 실패 후 어떻게 처리하는가	격리 수준 변경과 재시도 검증

낙관적 방식이라면 “포트폴리오에 버전 필드를 추가한다”에서 설계가 끝나지 않는다. 실제로 수정하는 것은 자식 이력인 경우가 많다. 부모를 직접 바꾸지 않는 두 작업도 충돌로 인식하도록 공통 버전을 갱신해야 하고, 충돌한 요청을 사용자에게 어떻게 알리거나 재시도할지도 정해야 한다. 이번 비교에서 그 정책까지 구현하고 성능을 측정한 것은 아니다.

Serializable 역시 적용 범위와 실패 처리까지 함께 결정해야 한다. 직렬화 실패 시 어떤 단위를 재시도할지, 그 안에서 반복하면 안 되는 작업은 없는지 확인할 일이 늘어난다. 이 대안이 원천적으로 부적합해서 제외한 것은 아니다. 기존 구조에서 문제를 해결하는 데 필요한 변경 범위를 고려했다.

선택한 방법은 이미 있던 PortfolioRepository.findByIdForUpdate를 원장 조회 전에 사용하는 것이다. V2 거래 등록과 스냅샷 생성은 이미 포트폴리오를 잠근 뒤 이력을 읽고 있었다. V1과 수정·삭제 경로를 같은 규약에 맞추면 공통 보호 범위를 만들 수 있었다. DB 스키마와 API 형식을 바꾸지 않고 적용할 수 있다는 점도 선택 이유였다.

PostgreSQL에서 행 잠금은 충돌하는 다른 변경·잠금 요청을 기다리게 하며, 일반적으로 트랜잭션 종료까지 유지된다.[5] 다만 부모 행을 잠갔다고 자식 이력 전체가 자동으로 보호되는 것은 아니다. 이번 설계는 같은 포트폴리오를 변경하는 경로가 모두 먼저 같은 행을 잠근다는 애플리케이션 규약에 의존한다. 일부 경로가 이 규약을 건너뛰면 같은 정합성 보장을 기대할 수 없다.

6. 읽기 전에 잠그고, 변경이 끝날 때까지 유지하기

수정 커밋 6edf55be에서 보유 정정·삭제와 거래 수정·삭제는 getPortfolio 대신 getPortfolioForUpdate를 호출하도록 바뀌었다. 기존 포트폴리오를 사용하는 V1 등록 경로에도 포트폴리오 잠금이 연결됐다. 핵심은 이 호출이 이력·보유 조회보다 앞에 있다는 것이다.[3]

아래는 import를 생략한 실제 저장소 메서드다.[4]

```
@Lock(LockModeType.PESSIMISTIC_WRITE)
@Query("select portfolio from Portfolio portfolio where portfolio.id = :id")
Optional<Portfolio> findByIdForUpdate(@Param("id") Long id);
```

거래 수정 흐름을 순서만 남겨 표현하면 다음과 같다. 전체 구현을 대체하는 코드는 아니다.

```
트랜잭션 시작
  포트폴리오 잠금 획득
  변경할 거래와 계산에 필요한 이력·보유 조회
  거래 수정 → 원장 재생 → 현재 보유 반영
  영향받는 날짜부터 스냅샷 갱신
커밋 또는 롤백 → 잠금 해제
```

재현 테스트의 원장 변경 트랜잭션은 격리 수준을 따로 지정하지 않아 PostgreSQL 16의 기본값인 Read Committed를 사용했다. 이 수준에서는 각 조회문이 시작할 때 이미 커밋된 데이터를 볼 수 있다. 둘째 요청의 이력 조회는 첫 요청이 커밋하고 포트폴리오 잠금을 놓은 뒤 시작하므로, 첫 거래의 12주를 읽고 자신의 20주를 더해 32주를 계산한다.[2][3][7]

잠금 순서도 맞췄다. 회원 잠금이 필요한 V1 등록은 회원 → 포트폴리오 순서를 따른다. 아직 포트폴리오 행이 없는 최초 생성은 회원 잠금으로 중복 생성을 막는다. 수정·삭제처럼 포트폴리오만 잠그는 경로는 그 뒤에 회원 잠금을 추가 획득하지 않는다. 서로 다른 순서로 잠가 상대가 가진 자원을 기다리는 상황을 만들지 않기 위해서다.

V2 등록과 예약 스냅샷 생성도 같은 포트폴리오 잠금 규약에 참여한다. 보호 범위는 하나의 HTTP 메서드나 한 엔티티의 저장 호출이 아니라, 공통 원장을 읽어 파생 데이터를 만드는 작업 전체다. 새 변경 API를 추가할 때도 기존 경로와 같은 순서로 보호되는지 확인해야 한다.

7. 외부 환율 조회를 잠금 밖으로 옮긴 이유

잠금을 앞에서 얻으면 정합성에 필요한 구간을 보호할 수 있지만, 그 안에서 기다리는 작업도 함께 늘어날 수 있다. 미국 자산을 평가하려면 환율이 필요하다. PR #583 리뷰 반영 기록에서는 외부 환율 조회와 잠금 범위를 함께 조정했다.[1][8]

환율을 가격마다 REQUIRES_NEW로 조회하면 원장 변경 트랜잭션이 가진 커넥션 외에 추가 커넥션이 필요하다. 외부 HTTP 응답을 기다리는 동안에도 회원·포트폴리오 잠금은 유지될 수 있다. 같은 포트폴리오의 다른 요청은 계산 자체에 필요한 시간뿐 아니라 외부 서비스의 응답 시간까지 기다리게 된다.

변경된 거래 등록·수정·삭제와 예약 스냅샷 생성은 필요한 날짜의 환율을 먼저 준비한다. 이후 TransactionTemplate으로 트랜잭션을 시작해 잠금을 얻고 이력·보유·스냅샷을 변경한다. 준비한 환율 맵을 계산에 전달하며, 환율 서비스의 Propagation.NEVER는 활성 트랜잭션 안에서의 호출을 거부한다. 당시 수정의 목적은 잠금 중 외부 통신을 피하면서 원장 변경의 원자성을 유지하는 것이었다.

사전 조회를 했다고 원장까지 그때 읽은 상태를 사용해서는 안 된다. 환율을 준비하는 동안 다른 거래가 커밋할 수 있기 때문이다. 환율 준비와 원장 변경의 근거를 분리하고, 이력·보유는 잠금 획득 뒤 다시 읽는다. 준비 중 원장에 날짜가 추가되어 환율 맵에 필요한 값이 없을 수도 있다는 점은 별도 정책으로 처리한다.

ADR은 실패와 누락을 다음처럼 구분한다. 외부 통신 오류는 원장 변경 전에 전파해 기존 데이터를 유지한다. 빈 환율 결과나 종가 누락으로 평가할 수 없는 날짜는 기존 스냅샷을 보존한다. 그 날짜의 보유가 실제로 비어 있다면 스냅샷을 삭제한다. 가격이나 환율이 준비되면 그 날짜를 포함하는 다음 원장 변경, 예약 스냅샷 작업, 또는 종가 동기화 후 갱신에서 다시 계산할 수 있다. 예약 작업과 종가 동기화 후 갱신은 기존 스냅샷 날짜도 검사한다. 재계산이 성공하기 전까지 조회에는 마지막으로 저장한 평가가 남는다.[1][9]

따라서 “거래가 성공하면 모든 날짜의 평가도 최신이다”라고 일반화할 수 없다. 32주·3,200원을 검사하는 핵심 재현은 필요한 가격이 준비된 조건이다. 잠금으로 원장 변경의 순서를 맞추는 문제와, 외부 데이터가 없는 날짜의 평가 정책은 연결되어 있지만 구분해서 설명해야 한다.

8. 무엇으로 수정 효과를 확인했는가

수정의 효과는 응답 성공률보다 데이터가 다시 일치하는지로 확인했다. ADR의 2026년 10월 20일 미국 거래 연동 브랜치 기록에서, 수정 전 동시성 테스트는 20개 중 13개가 실패했다. 핵심 동시 정정은 10회 모두 원장 32주·보유 30주였다. 정정과 등록을 겹치면 27주 대신 25주, 정정과 삭제를 겹치면 12주 대신 10주가 저장된 사례도 기록했다.[1]

수정 후에는 같은 동시성 인수 테스트 20개를 완화하지 않고 통과했다. 집중 검증이 보호하는 관계는 다음과 같다.[1][2]

시나리오	확인하는 결과
서로 다른 거래 동시 정정, 10회 반복	원장·보유 32주, 스냅샷 3,200원
정정 + 삭제, 정정 + V1/V2 등록	경로가 달라도 최종 원장과 파생 데이터 일치
20주 보유에서 15주 매도 두 건	한 건만 성공, 거절 요청 롤백, 5주 유지
보유 정정·삭제 + V2 등록	이력과 현재 보유 일치, 과거 평가 보존
정정 + 스냅샷 생성	커밋된 원장 반영, 스냅샷 중복 없음
특정 회원의 포트폴리오 잠금 대기	다른 회원의 정정은 먼저 완료 가능
뒤 날짜 스냅샷 저장 실패	거래·보유·앞 날짜 평가 변경도 롤백

다른 회원의 독립성 검증은 잠금 범위가 전역으로 넓어지지 않았는지 확인한다. 롤백 검증은 뒤 날짜 평가 금액이 저장 범위를 초과하게 만들어, 앞서 수행한 거래 정정과 현재 보유 변경까지 남지 않는지 검사한다. 성공 사례와 실패 사례를 함께 확인해야 원장과 파생 데이터가 하나의 변경 단위라는 주장을 뒷받침할 수 있다.

전체 검증 기록의 시점도 구분할 필요가 있다. 10월 19일 기록은 전체 759개 중 757개 통과·1개 실패·1개 건너뜀이었다. 별도 리그 테스트의 날짜 의존 실패가 남아 있어 전체 통과로 기록하지 않았다. 10월 20일 연동 브랜치의 전체 검증은 894개 중 893개 통과·1개 건너뜀·실패 0개, spotlessCheck 통과로 기록됐다.

이 수치는 당시 ADR의 실행 기록이며, 현재 모든 기능의 무결성이나 운영 처리량을 보장하는 수치가 아니다. 이번 글 작성에서는 테스트를 재실행하지 않았다. 재현을 다시 확인할 때는 PostgreSQL Testcontainers 실행 환경에서 ./gradlew test --tests '*PortfolioConcurrencyAcceptanceTest'를 사용하고, 그 실행의 로그를 별도로 남겨야 한다.

9. 이 선택의 비용과 다음에 확인할 것

비관적 잠금은 같은 포트폴리오 변경을 순서대로 처리하는 비용을 만든다. 과거 거래를 수정해 재계산할 날짜가 많아지면 잠금 유지 시간과 뒤 요청의 대기 시간도 길어질 수 있다. 이번 검증은 특정 경합의 정합성을 확인했다. 처리량이나 잠금 대기 p95를 측정해 다른 대안보다 빠르다고 결론 낸 실험은 아니다.

또한 부모 행 잠금은 모든 조회 API에 일관된 시점의 결과를 보장하지 않는다. 잠금 없이 별도로 읽는 여러 데이터가 어떤 시점을 나타내야 하는지는 조회 계약에서 따로 다뤄야 한다. 이미 잘못 저장된 운영 데이터의 복구도 이번 수정의 범위 밖이다. 새 원장 변경 경로가 잠금 규약을 지키는지 확인하는 일과, 기존 불일치 데이터를 찾아 복구하는 일은 각각 필요하다.

이 경험에서 가져갈 기준은 세 가지다. 먼저 지켜야 하는 데이터 관계를 식으로 적고, 그 계산에 필요한 첫 읽기보다 앞에서 공통 보호 구간을 시작한다. 다음으로 같은 결과를 바꾸는 모든 진입점이 그 구간에 참여하는지 확인한다. 마지막으로 테스트에서 요청 중첩을 관측하고, 응답뿐 아니라 최종 데이터와 롤백 결과까지 비교한다.

DAYNOMY에서는 원장 32주·보유 30주라는 구체적인 실패가 잠금의 시점을 다시 보게 했다. 이후 잠금 위치뿐 아니라 외부 환율 조회와 실패 정책까지 경계를 정리했다. 다음 동시성 문제에서도 어떤 잠금 기술을 쓸지에 앞서, 어떤 값을 읽어 함께 계산하고 저장하는지부터 확인하려고 한다.

근거 자료

저장소 자료는 작성일 확인한 커밋 075b8ed0에 고정했다. 수정 이력은 별도 커밋으로 연결했다. 외부 문서는 개념 설명의 근거이며 프로젝트가 해당 명세 버전을 사용한다는 의미는 아니다.

[1] DAYNOMY ADR: 포트폴리오 원장 변경 잠금 범위 — 재현 조건, 대안, 10월 19~20일 검증 기록과 한계.
[2] PortfolioConcurrencyAcceptanceTest — overlap, awaitBlockedRequests, assertConsistent와 경합 시나리오.
[3] 원장 변경 전 잠금 수정 커밋 6edf55be — 작성자 choiyoung69, 변경 전후 서비스 흐름.
[4] PortfolioRepository — findByIdForUpdate의 비관적 쓰기 잠금.
[5] PostgreSQL 16: Explicit Locking — 행 잠금의 충돌·유지 기간·교착 상태.
[6] Jakarta Persistence: EntityManager — 영속성 컨텍스트의 엔티티 식별자와 관리 객체.
[7] PostgreSQL 16 · Read Committed — 기본 격리 수준과 조회문별 가시성.
[8] SavedPortfolioService — 환율 준비 후 트랜잭션 시작, 잠금 후 원장 변경과 스냅샷 갱신.
[9] PortfolioSnapshotService — 기존 스냅샷 날짜 재검사와 종가 동기화 후 갱신.